

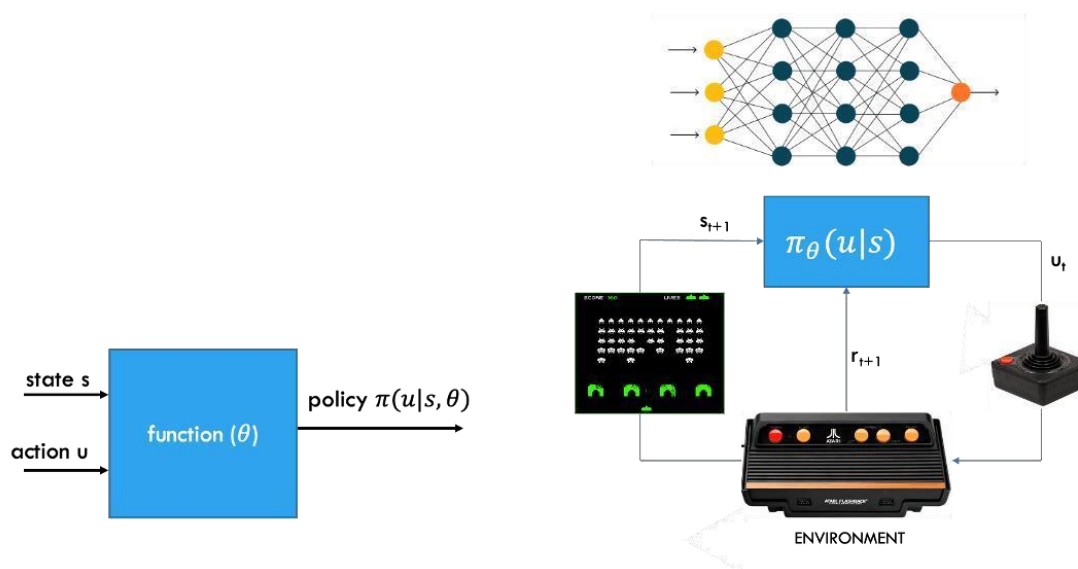
Reading 6.5: Policy Gradient

In Q-learning we must calculate the max-term over all Q-values, given a state, and this can be a problem in continuous, very high dimensional state-action spaces. What if we could parametrise the policy and learn an optimal policy directly?

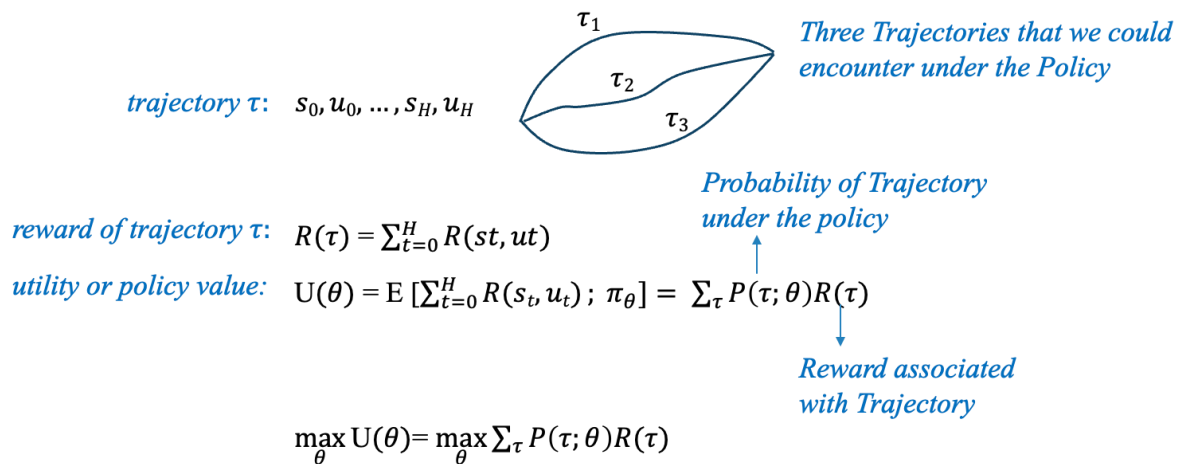
Welcome to this lesson where we unravel the complexities of Policy Gradient. After reading through the document you will:

- Understand the foundational idea behind Policy Gradient
- Learn how Policy Gradient works

In policy gradient, we learn a policy function, parametrised by θ , that outputs a distribution over actions that you want to take: $\pi(u|s, \theta)$. In the context of an LLM, this translates into a distribution over 'next words'. Going forward we will use u to represent the action instead of a . We will assume that $\gamma = 1$.



We look for a stochastic policy as a deterministic policy changes too abruptly and could mess up our gradient descent process. Let's introduce the trajectory τ as a state-action sequence.



We are looking to find the weights θ that maximise the policy value. Let's find the gradient:

$$\nabla_{\theta} U(\theta) = \nabla_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau) \longrightarrow \nabla_{\theta} U(\theta) = \sum_{\tau} \nabla_{\theta} P(\tau; \theta) R(\tau)$$

$$\nabla_{\theta} U(\theta) = \sum_{\tau} P(\tau; \theta) \frac{\nabla_{\theta} P(\tau; \theta) R(\tau)}{P(\tau; \theta)}$$

$$\nabla_x \log(f(x)) = \frac{\nabla_x f(x)}{f(x)} \longrightarrow \nabla_{\theta} U(\theta) = \sum_{\tau} P(\tau; \theta) \nabla_{\theta} \log P(\tau; \theta) R(\tau)$$

$$= \sum_{\tau} P(\tau; \theta) \nabla_{\theta} \log P(\tau; \theta) R(\tau)$$

$$\approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

The formula for $\hat{g}$ represents a (statistical) expectation and we can approximate it with a sample-based estimate of that expectation. We sample m paths under the policy π_{θ} . The probability of good paths (positive reward) will be increased, while the inverse will happen with bad paths (negative reward).

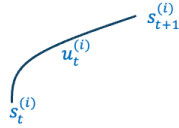
Let's decompose the path in states and actions:

The trajectory $\tau^{(i)}$ consists of states $s_t^{(i)}$, actions $u_t^{(i)}$, and transitions. The probability of the trajectory is defined as:

$$P(\tau^{(i)}; \theta) = \prod_{t=0}^H P(s_{t+1}^{(i)} | s_t^{(i)}, u_t) \cdot \pi_{\theta}(u_t^{(i)} | s_t^{(i)})$$

- $P(s_{t+1}^{(i)} | s_t^{(i)}, u_t)$: Transition probability under the dynamics model.
- $\pi_{\theta}(u_t^{(i)} | s_t^{(i)})$: Policy's probability of taking action $u_t^{(i)}$ given state $s_t^{(i)}$.

What is the probability of a path?



$$\begin{aligned} \nabla_{\theta} \log P(\tau^{(i)}; \theta) &= \nabla_{\theta} \log \left[\prod_{t=0}^H P(s_{t+1}^{(i)} | s_t^{(i)}, u_t) \cdot \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \right] \\ &= \nabla_{\theta} \left[\sum_{t=0}^H \log P(s_{t+1}^{(i)} | s_t^{(i)}, u_t) + \sum_{t=0}^H \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \right] \\ &\quad \text{No } \theta \\ &= \nabla_{\theta} \sum_{t=0}^H \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \\ &= \sum_{t=0}^H \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \quad (\text{no dynamics model required!!}) \end{aligned}$$

unbiased estimate of gradient

$$\approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) R(\tau^{(i)})$$

The formula confirms that we do not need to understand the dynamics of the model to calculate our gradient. The training process however is noisy (high variance) and lots of samples are required to get a good estimate.

There are a few options to make the Policy Gradient formula practical in a real-world situation:

1. Baseline
2. Temporal Structure
3. KL Divergence Trust Region

Baseline b

If you have 3x positive reward you increase probability of all 3. Ideally, we alternate between + and - updates of $\hat{g}$, hence we introduce a baseline b. Introducing a baseline will make sure that we need less samples to get a good estimate for $\hat{g}$. We basically want to increase probability of paths that are better than average and decrease the probability of paths that are worse than average. We can measure what average (b) is as we sample.

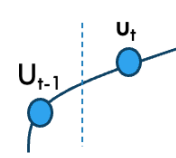
The Policy Gradient formula allows us to compute an estimate of the gradient without knowing the exact dynamics of the environment (how the state transitions happen), as long as we can observe rewards and actions. The key here is that this estimate is unbiased, meaning that, over time, this method will converge to the true gradient, ensuring we're updating the policy in the right direction. In summary, when introducing a baseline b , we reduce variance and remain unbiased.

Temporal Structure

We are only interested updating $\hat{g}$ with actions that come with future rewards that come after taking that action. We start from $t=0$ and end at $t=H-1$ (H time steps) and removing terms that do not depend on the current and future actions to lower the variance of $\hat{g}$.

$$\begin{aligned}\hat{g} &= \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log P(\tau^{(i)}; \theta) \left(R(\tau^{(i)}) - b \right) \\ &= \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{H-1} \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \left[R(s_t^{(i)}, u_t^{(i)}) - b \right] \\ &= \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{H-1} \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \left[\sum_{k=0}^{t-1} R(s_k^{(i)}, u_k^{(i)}) + \sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)}) - b \right]\end{aligned}$$

only interested on
current and future
actions and rewards



$$\hat{g} = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{H-1} \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \left[\sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)}) - b(s_t^{(i)}) \right]$$

The baseline $b(s_t^{(i)})$ is chosen as an approximation of the expected reward. The expected reward is represented by the value function $V_{\pi}(s_t)$ under the current policy π . We define the advantage function A as:

$$A(s_t, u_t) = \sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)}) - V_{\pi}(s_t)$$

Our Policy Gradient algorithm becomes:

$$\hat{g} = \frac{1}{m} \sum_{l=1}^m \sum_{t=0}^{H-1} \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) A(s_t^{(i)}, u_t^{(i)})$$

This equation represents the gradient estimator used in policy gradient methods for reinforcement learning. It estimates the gradient of the expected cumulative reward with respect to the policy parameters θ .

1. **Outer Summation** ($\frac{1}{m} \sum_{i=1}^m$):
 - Averages over m episodes (or trajectories) sampled from the environment.
2. **Summation over Time** ($\sum_{t=0}^{H-1}$):
 - Accumulates contributions from all time steps t in a single trajectory of length H (horizon).
3. **Policy Gradient** ($\nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)})$):
 - Computes the gradient of the log-probability of the action u_t (given state s_t) with respect to the policy parameters θ .
4. **Return Term** ($\sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)}) - b(s_t^{(i)})$):
 - Represents the **advantage function**, which measures how much better or worse the action u_t is compared to a baseline $b(s_t)$. This reduces variance in the gradient estimation.

KL Divergence Trust Region

Policy Gradient is a cornerstone in reinforcement learning, offering a robust approach to handling continuous, high-dimensional action spaces. Its unbiased gradient estimates and practical enhancements, like baselines and temporal structure adjustments, make it a powerful tool for realworld applications.

Next, we will look at the 3rd option to reduce the variance of the policy gradient estimates: KL Divergence Trust Region.

KL Divergence Trust Region ensures stability during the policy update step by constraining how much the policy can change between iterations. Popular algorithms include Trust Region Policy Optimisation, TRPO, and Proximal Policy Optimisation or PPO

Try it Yourself

Challenge yourself to thoroughly understand every step of the policy gradient formula derivation outlined in this document.
